

15 May 2026

DEEP AND REINFORCEMENT LEARNING

Project Presentation

João Proença 202207835
Joana Pimenta 202206120

Francesco Renna
Tânia Maria Pereira Lopes

OUR CONTENTS

DISCRIMINATIVE

01. Problem & Approach Overview

02. Model Architecture & Design Choices

03. Training Strategy & Iterative Improvements

04. Results

05. Conclusions

GENERATIVE

01. Problem & Approach Overview

02. Model Architecture & Design Choices

03. Results

04. Conclusions

DISCRIMINATIVE

PROBLEM & APPROACH OVERVIEW

The Dataset: DeepFakeFace (HuggingFace) Four categories tested as separate experiments:

- wiki → Real faces
- inpainting → Fakes generated via inpainting
- insight → Fakes via face-swap (InsightFace)
- text2img → Fakes generated from text prompts
- all_fakes → A combined category mixing all three fake types

The Problem: Each generator produces fundamentally different artifacts so rather than just asking "can we detect fakes?", we ask "are all fakes equally detectable?" Training and evaluating separately per generator probes each technique's detectability, while the all_fakes experiment tests generalization across all three.

PROBLEM & APPROACH OVERVIEW

The Models Considered:

Model	Type	Pretrained
NN_1	Fully-connected baseline	✗
ResNet34	CNN + skip connections	✓
EfficientNet	CNN + compound scaling	✓
Xception	CNN + depthwise separable convs	✓
ViT_B_16	Vision Transformer	✓

The Transfer Learning Strategy (for all pretrained models):

- Frozen — only the classification head is trained
- Full — all layers unfrozen and fine-tuned

MODEL ARCHITECTURE & DESIGN CHOICES

NN_1

- Our starting point
- 4 FC layers: $12,288 \rightarrow 512 \rightarrow 256 \rightarrow 64 \rightarrow 1$, dropout at each layer
- A simple MLP on flattened pixels, no assumptions about spatial structure
- Establishes our lower bound: how much does spatial reasoning actually matter for detecting fakes?

ResNet34

- First step up: a CNN that learns local features through residual connections
- Chosen for its balance of depth and stability, deep enough to capture subtle artifacts, without the training instability of very deep networks
- Do skip connections and local feature extraction actually help over raw pixels?

EfficientNet-B0

- Scales depth, width and resolution jointly
- Lets us ask: is a more parameter-efficient architecture competitive, or does raw capacity win?

MODEL ARCHITECTURE & DESIGN CHOICES

Xception

- Depthwise separable convolutions decouple spatial and channel-wise learning
- Not just another CNN choice, this architecture has a proven track record specifically in deepfake detection literature
- A natural ceiling for what CNN-based approaches can achieve on this problem

ViT-B16

- Drops all CNN assumptions entirely, no local filters, no spatial inductive bias
- Splits the image into 196 patches and lets self-attention figure out which regions matter globally
- The key question: do deepfake artifacts require local texture analysis or global inconsistency reasoning?

TRAINING STRATEGY & ITERATIVE IMPROVEMENTS

Developing robust deepfake classifiers required multiple iterations to stabilize training, prevent overfitting, and ensure our models were learning genuine facial features rather than digital artifacts. Below are the main steps of the evolution of our training strategy.

Mitigating Overfitting and Bias

The Evolution of Learning Rates & Scheduling

Dataset Scaling & Class Balancing

Hardware & Speed Optimization

MITIGATING OVERFITTING AND BIAS

Data Augmentation

- **Primary Goal:** Mitigate overfitting and prevent the model from fixating on background noise or compression artifacts.
- **Application:** Exclusively applied to the training dataset.
- **Techniques Used:**
 - Random horizontal flips
 - $\pm 10^\circ$ rotations
 - Color jitter (adjusting brightness, contrast, saturation, and hue)
- **Result:** Forces the model to generalize and focus on the core facial structures rather than irrelevant details.

Early Stopping

- **Primary Goal:** Prevent the model from memorizing the training data.
- **Mechanism:** Continuously monitors the validation loss during the training process.
- **Key Parameters:**
 - Patience: 5 epochs
 - Minimum Delta: $1e-4$
- **Result:** Halts training if there is no meaningful improvement within the patience window, which saves computational resources and preserves the best model weights.

THE EVOLUTION OF LEARNING RATES & SCHEDULING

The Grad-CAM Insight

- **The Discovery:** Grad-CAM visualizations revealed that a 0.001 learning rate worked perfectly for frozen models (in a sense that used faces to decide), but caused "catastrophic forgetting" (destroying pre-trained weights) when applied to fully unfrozen models.
- **The Solution:** Implemented a split learning rate strategy based on the model's state.
- **Frozen Models:** Maintained the 0.001 learning rate.
- **Unfrozen (Full) Models:** Reduced the learning rate significantly to 0.0001 to protect the learned weights.

Refining the Scheduler

- **Initial Approach:** Used a StepLR scheduler (dividing the learning rate by 10 every 5 epochs).
- **The Problem:** This timeline-based reduction was too aggressive and stalled model progress prematurely.
- **The Solution:** Transitioned to a dynamic ReduceLROnPlateau scheduler, which adjusts the rate based on actual performance (validation loss) rather than a rigid schedule.
- **New Parameters:** The system now halves the learning rate (factor = 0.5) only after 3 full epochs of zero improvement (patience = 3).
- **Result:** Achieved much smoother and more stable model convergence.

DATASET SCALING & CLASS BALANCING

Scaling Up

- **Initial Phase:** Early rapid prototyping was done using subsets of 10,000 images per category for the all_fakes dataset.
- **The Scale-Up:** Expanded training to the full 30,000 images per category once sufficient computational power was secured.
- **Primary Goal:** Maximize the model's exposure to a wider diversity of generative methods, ensuring it learns a broader range of patterns.

Dynamic Loss Weighting

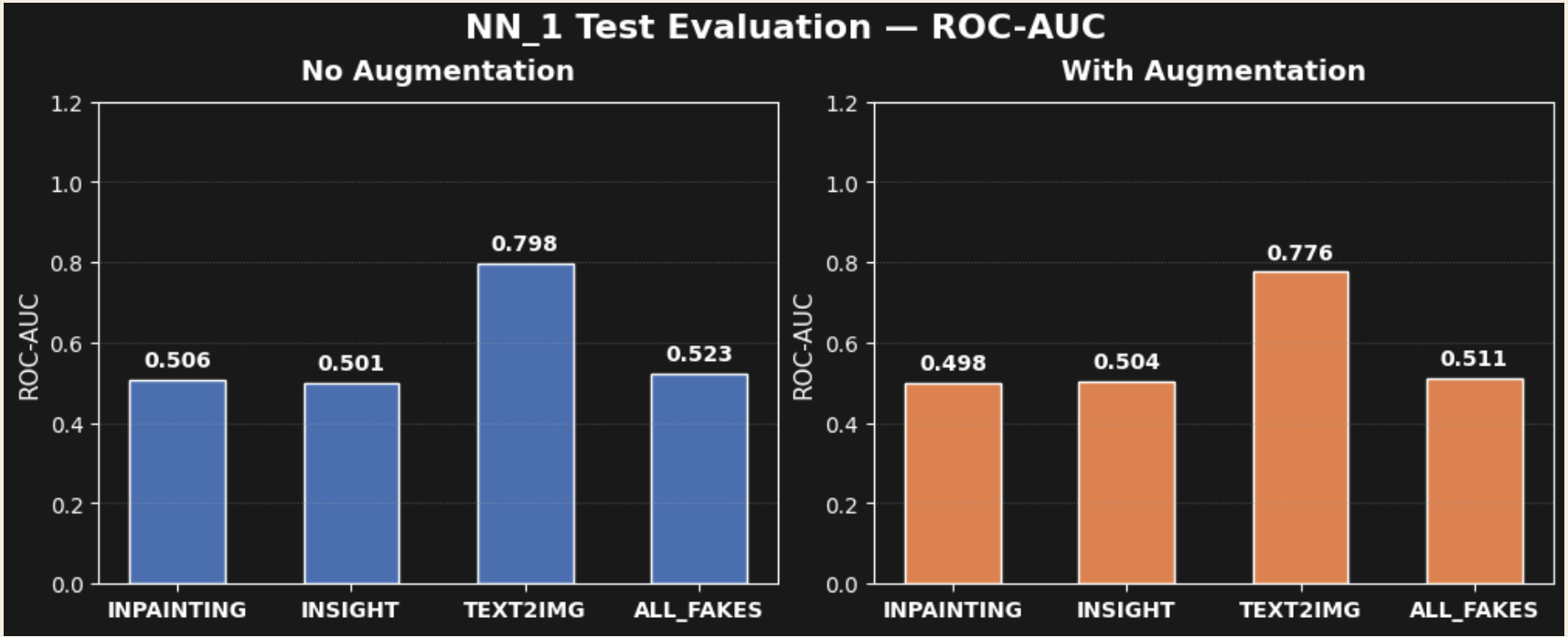
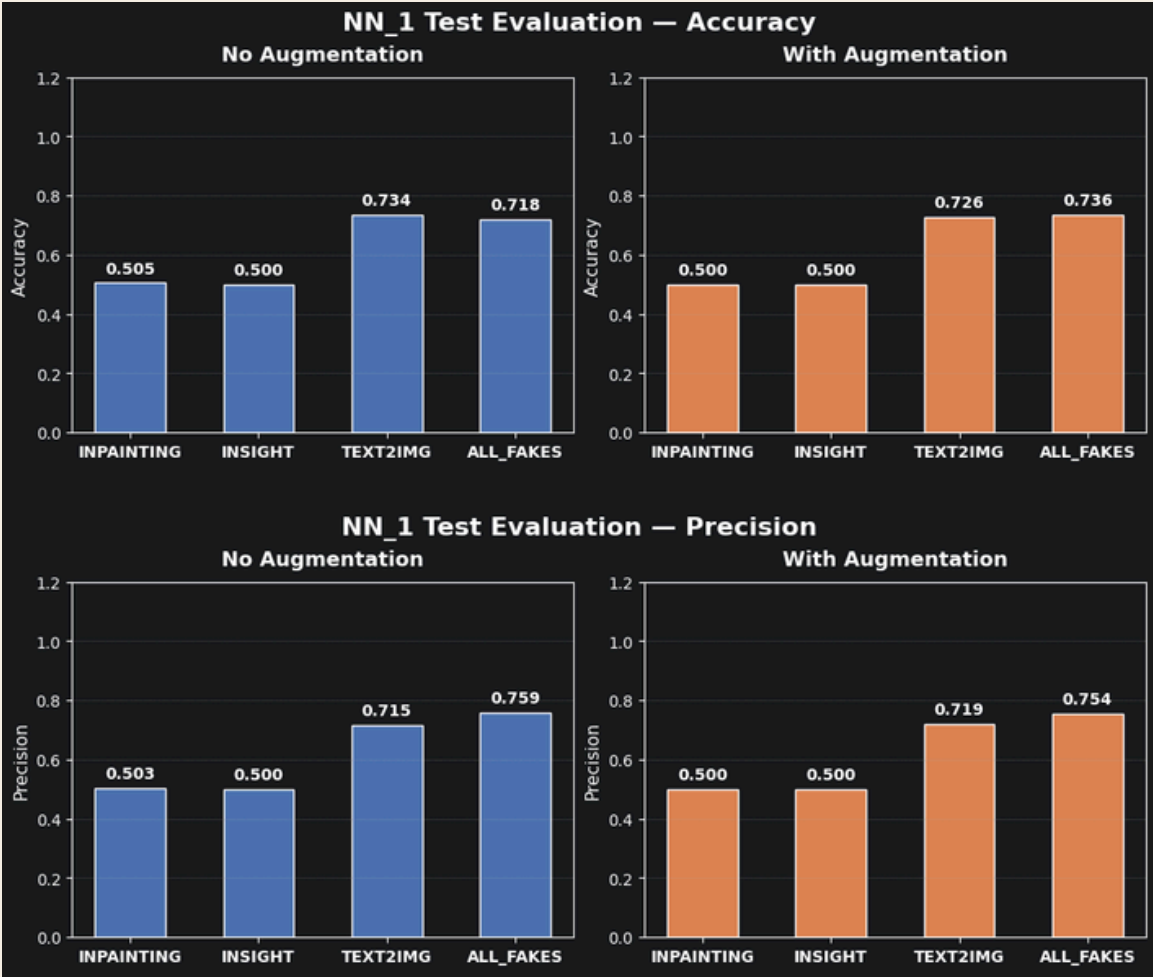
- **The Challenge:** Real and fake datasets frequently have unequal class distributions (imbalanced data).
- **The Mechanism:** Utilized the BCEWithLogitsLoss function paired with a dynamically calculated pos_weight tensor.
- **How it Works:** The tensor automatically calculates the exact ratio of real to fake samples present in the training set.
- **Result:** The system heavily penalizes errors made on the minority class. This forces the model to actually learn the distinguishing features rather than taking the easy route of simply guessing the majority class every time.

HARDWARE & SPEED OPTIMIZATION

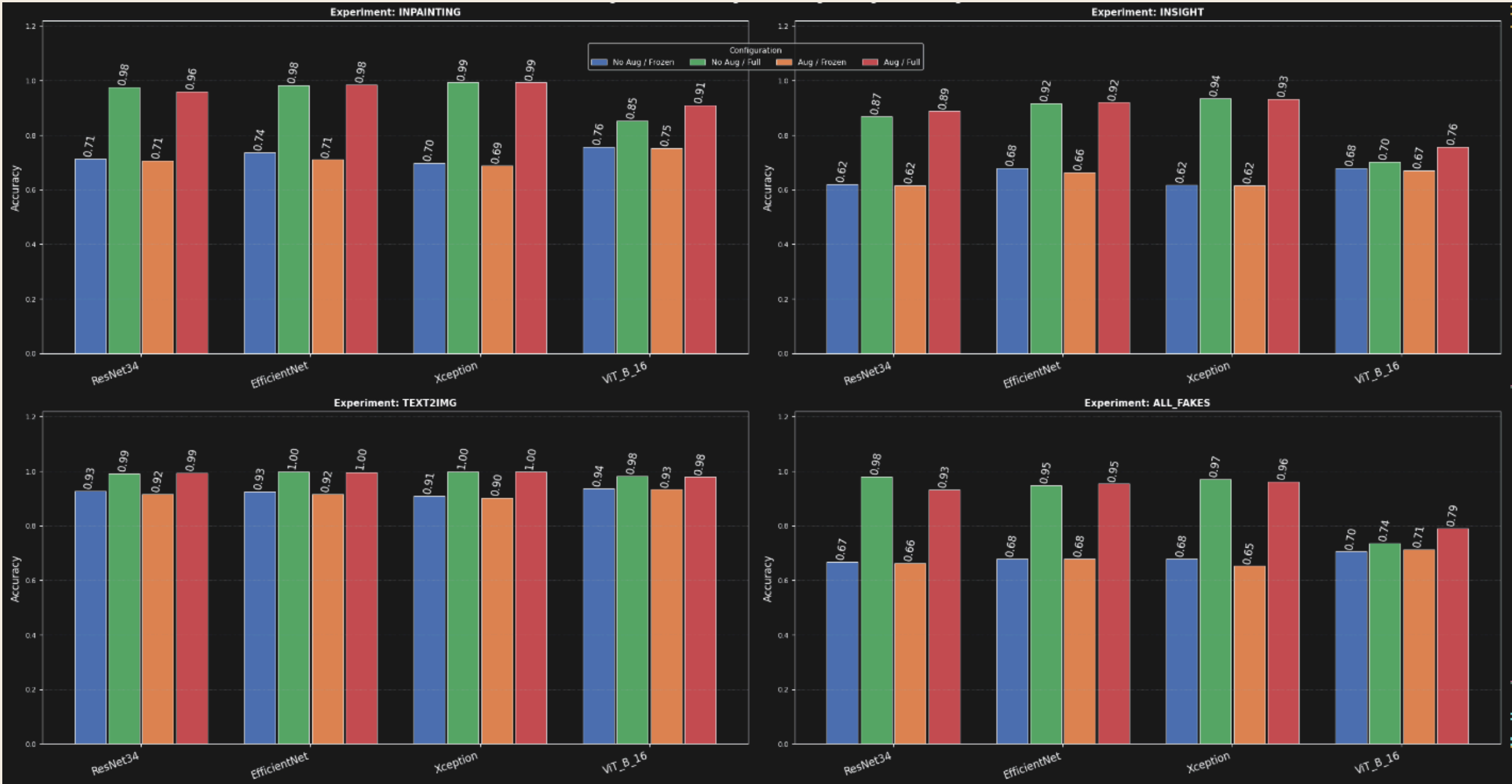
Dynamic Loss Weighting

To handle the increased dataset size efficiently without altering the underlying architecture logic, we leveraged PyTorch 2.0+'s compilation features. By simply wrapping our initialized architectures with `torch.compile(model)`, we achieved a "free" speedup during the forward and backward passes, drastically reducing our epoch turnaround times.

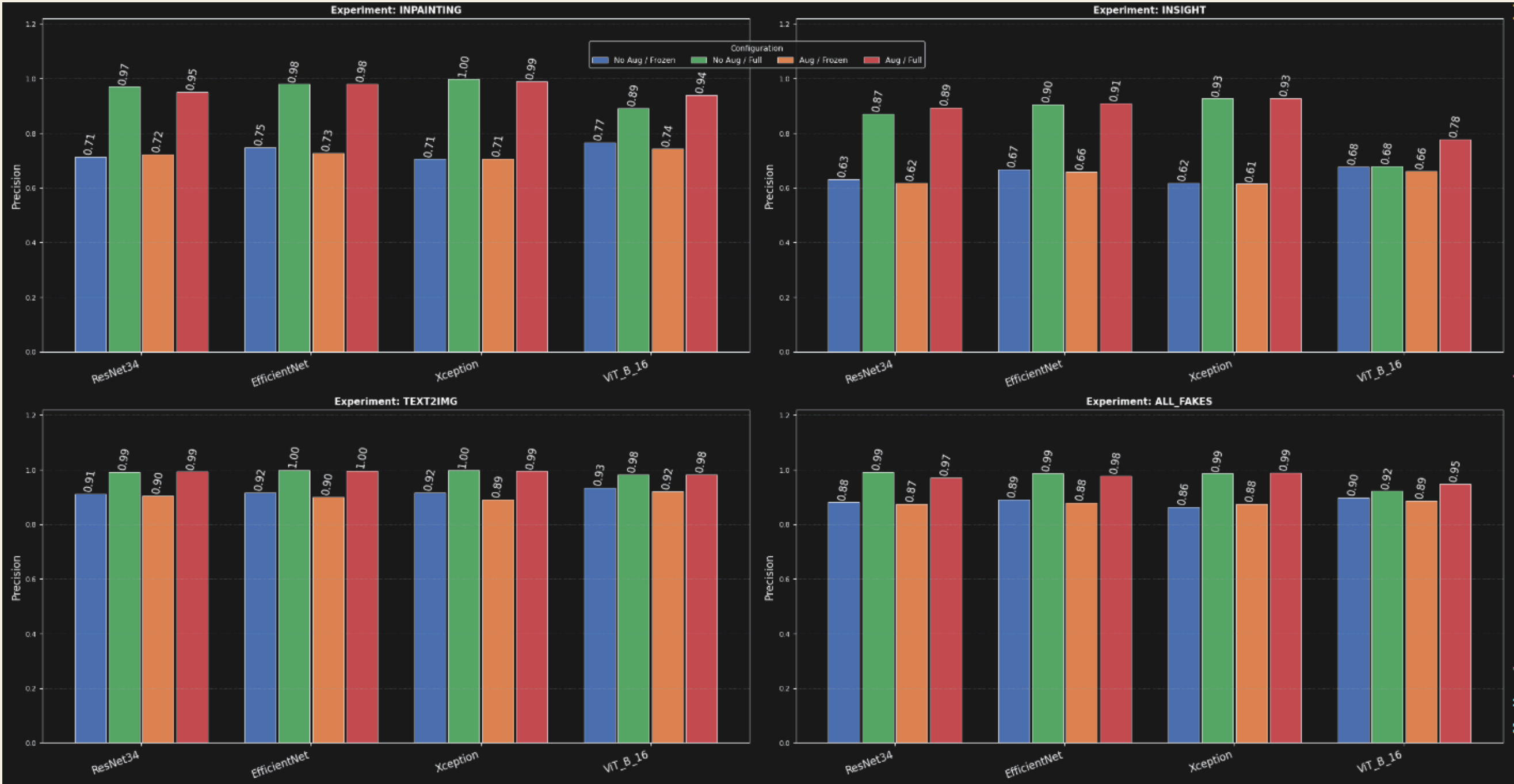
RESULTS NN_1 BASELINE



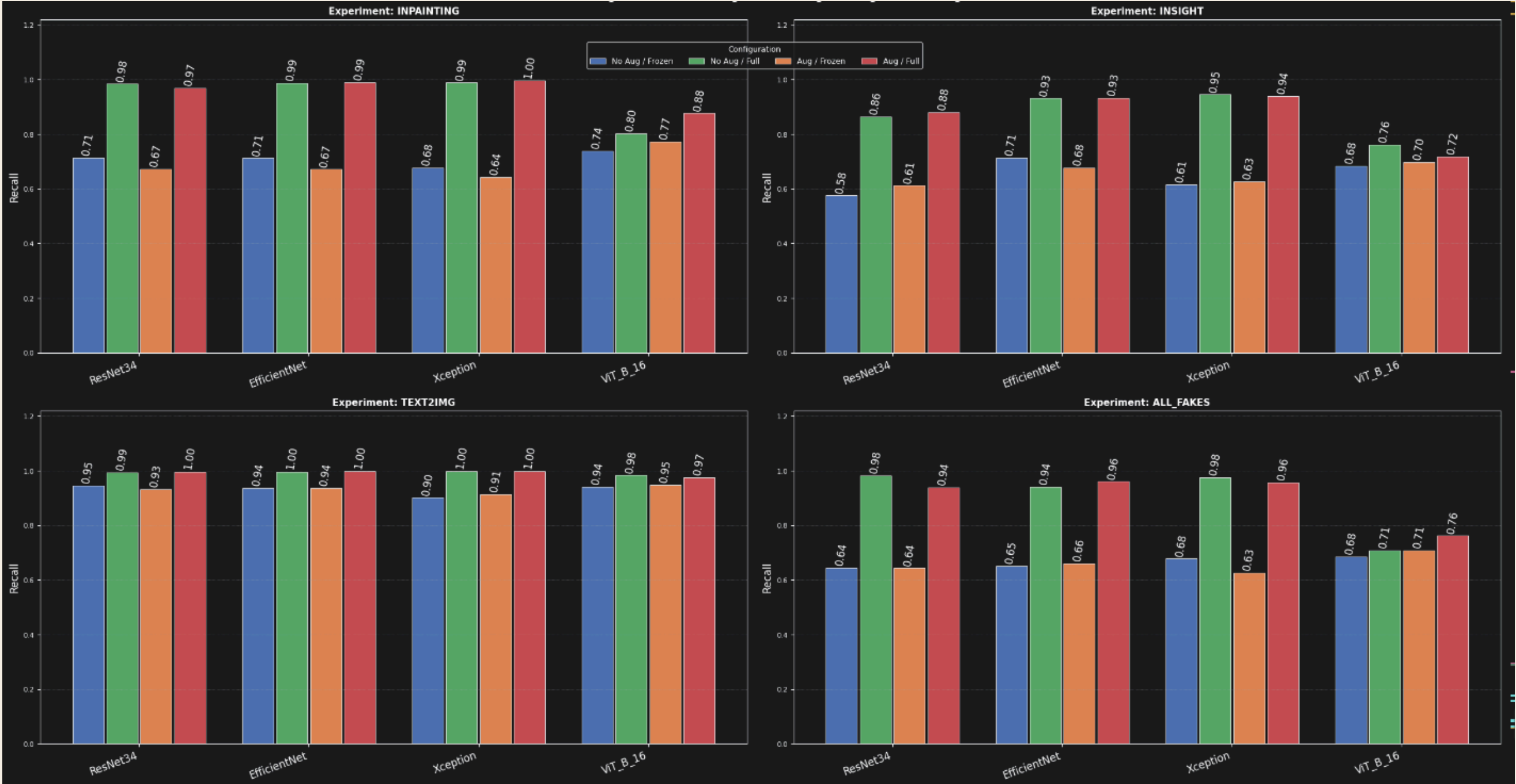
RESULTS ACCURACY



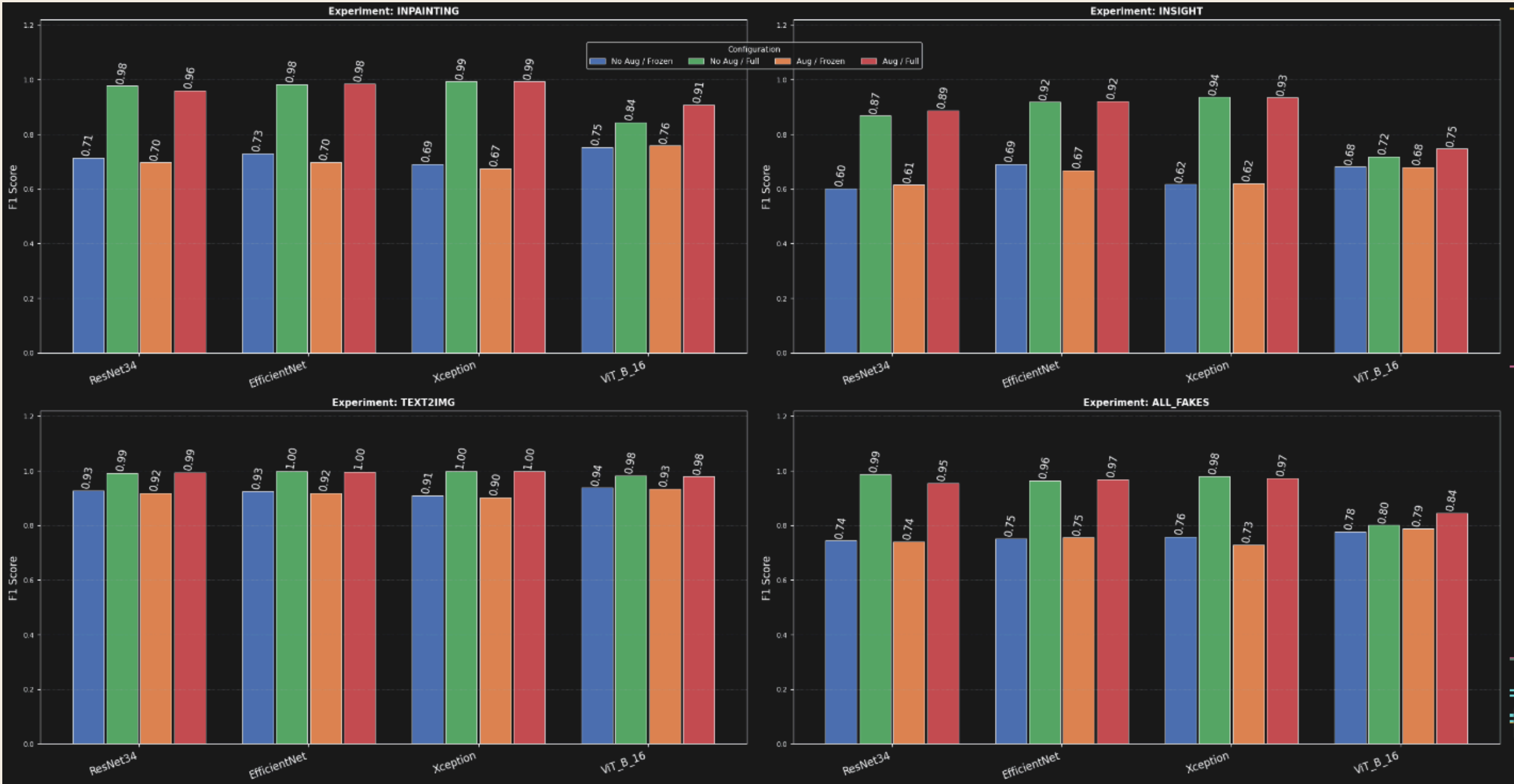
RESULTS PRECISION



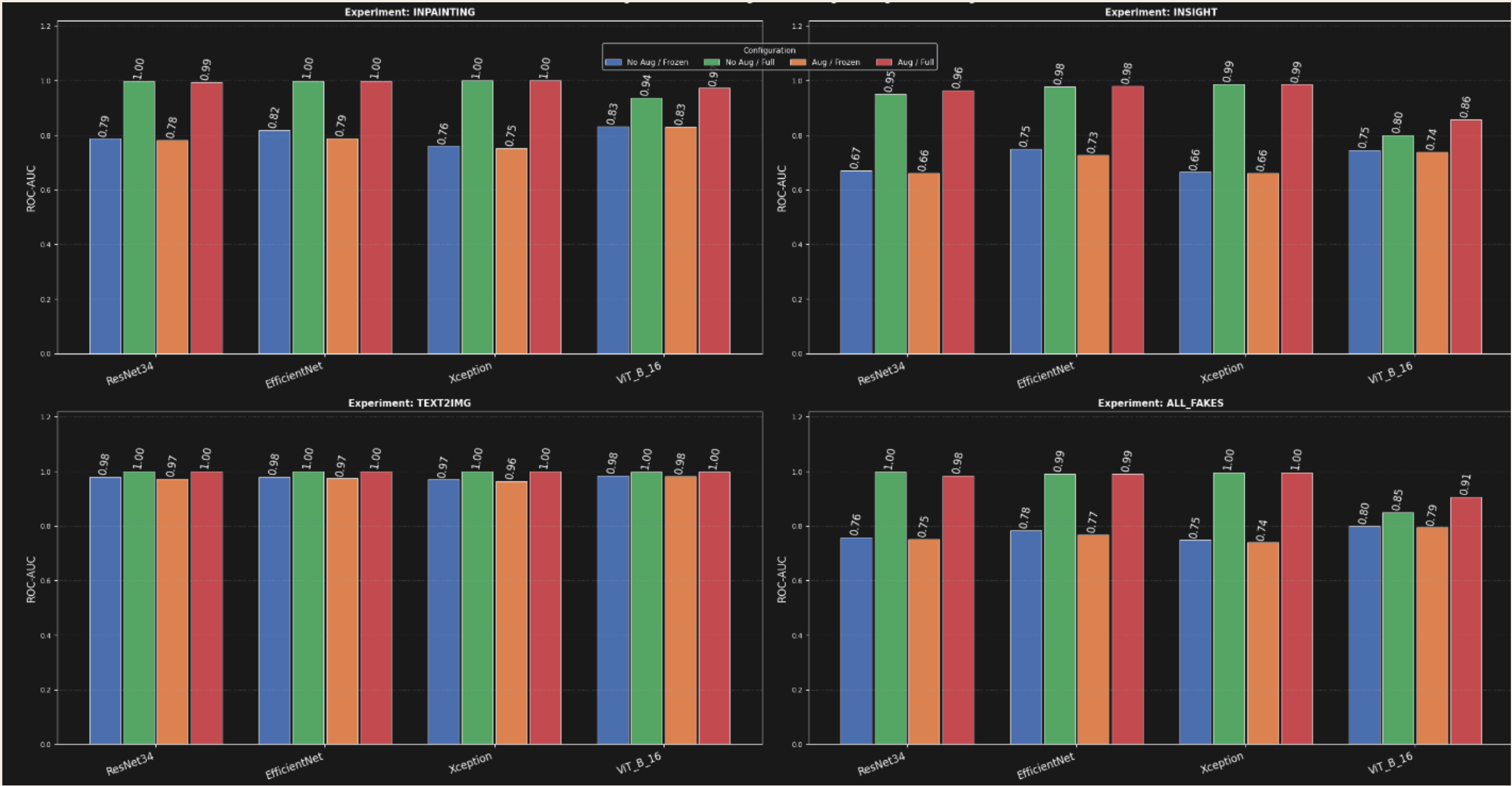
RESULTS RECALL



RESULTS F1-SCORE



RESULTS ROC-AUC

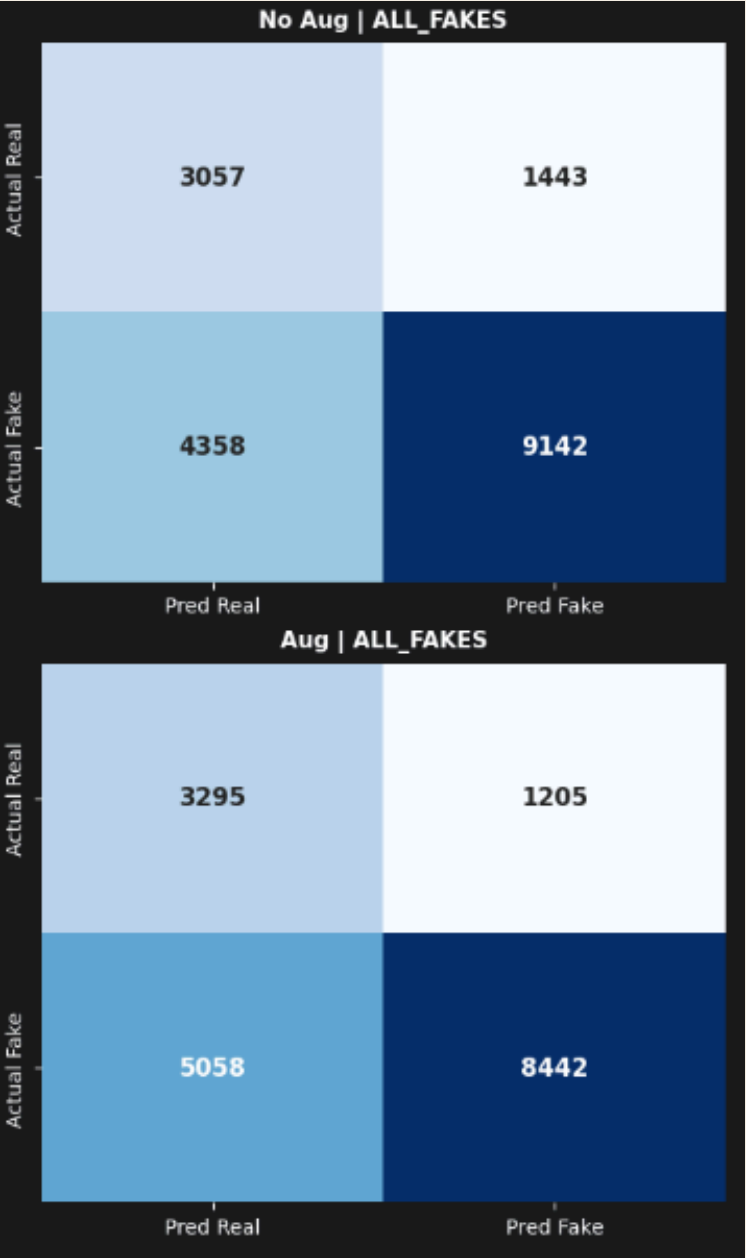


RESULTS AUGMENTATION EFFECT

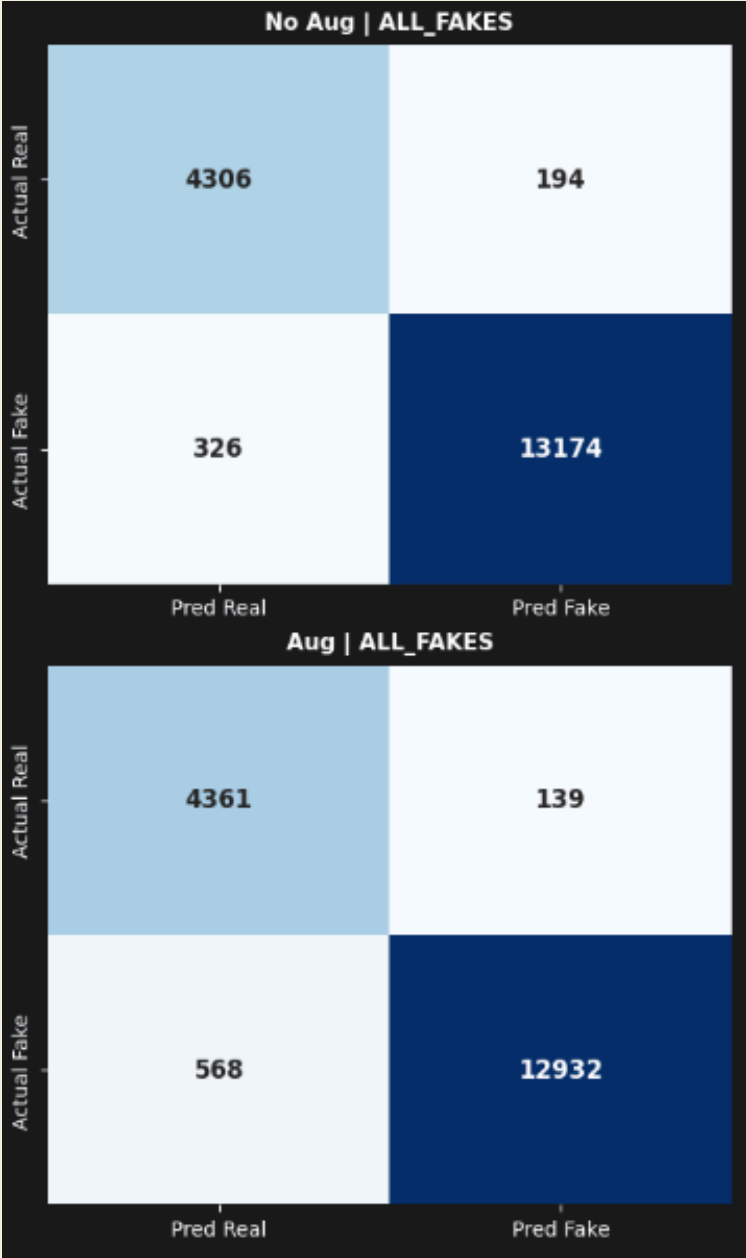


RESULTS AUGMENTATION EFFECT

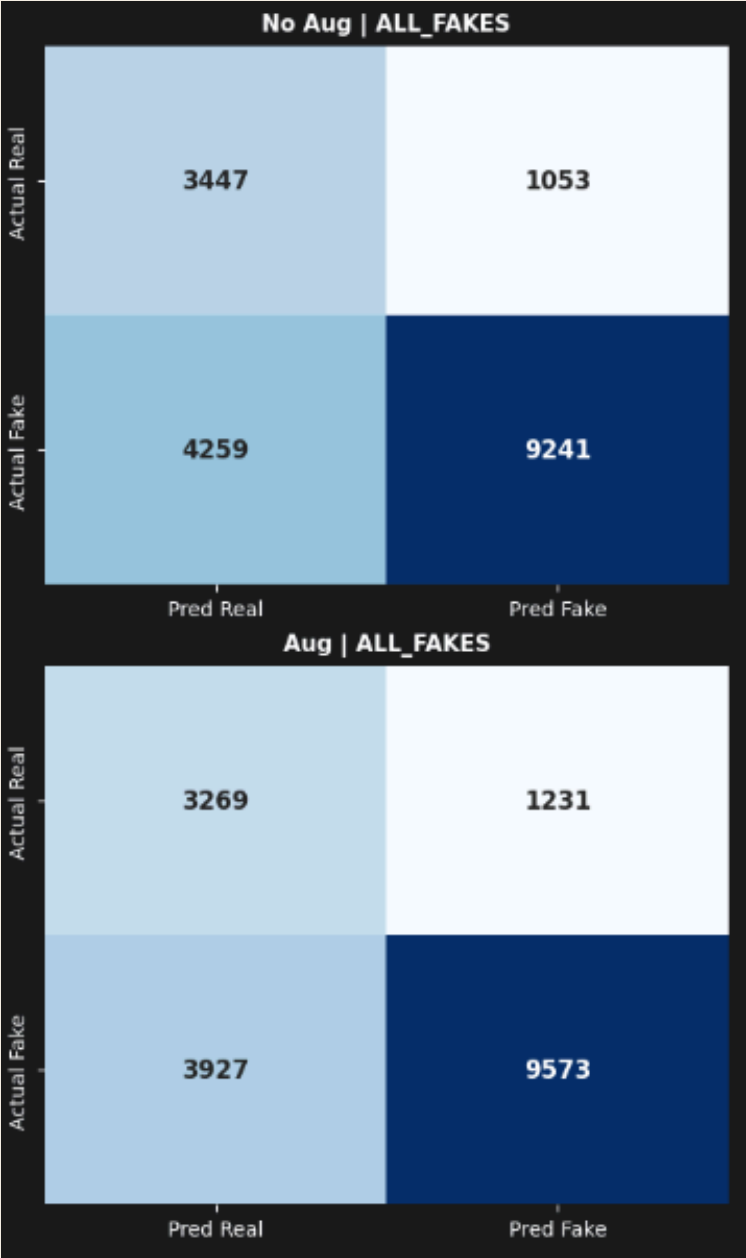
Xception Frozen



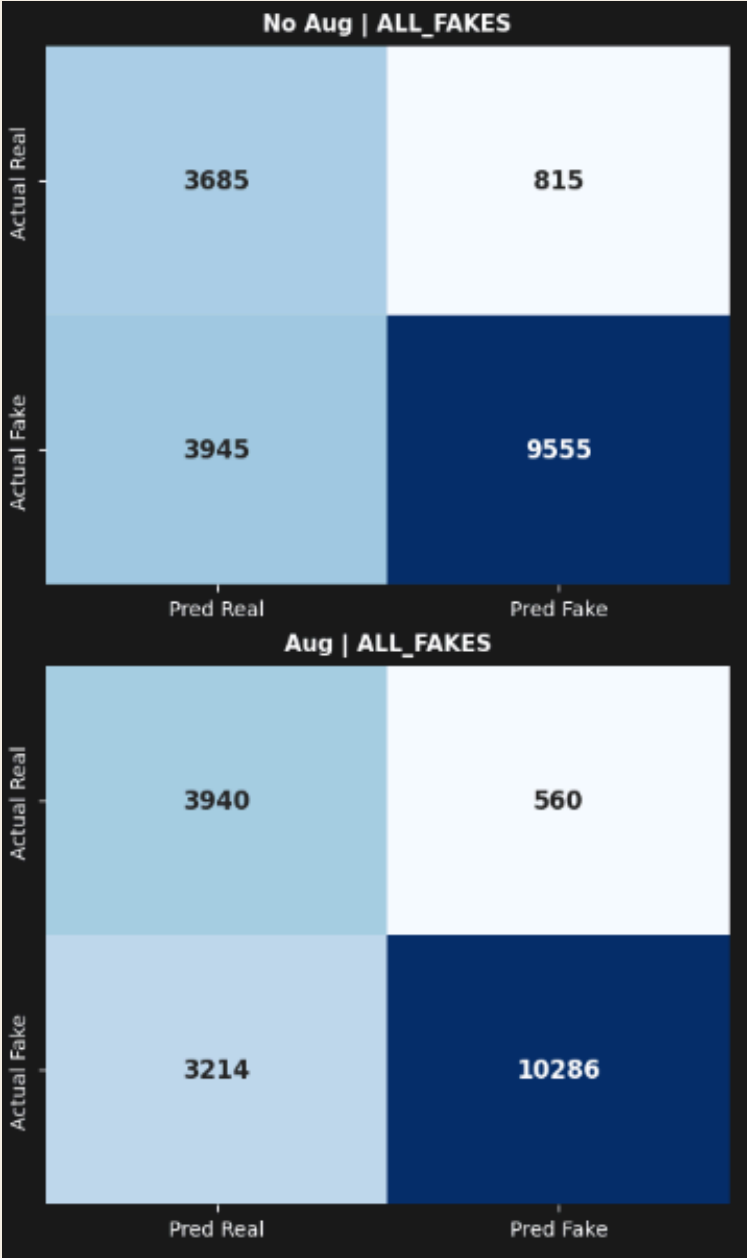
Xception Full



ViT_B_16 Frozen

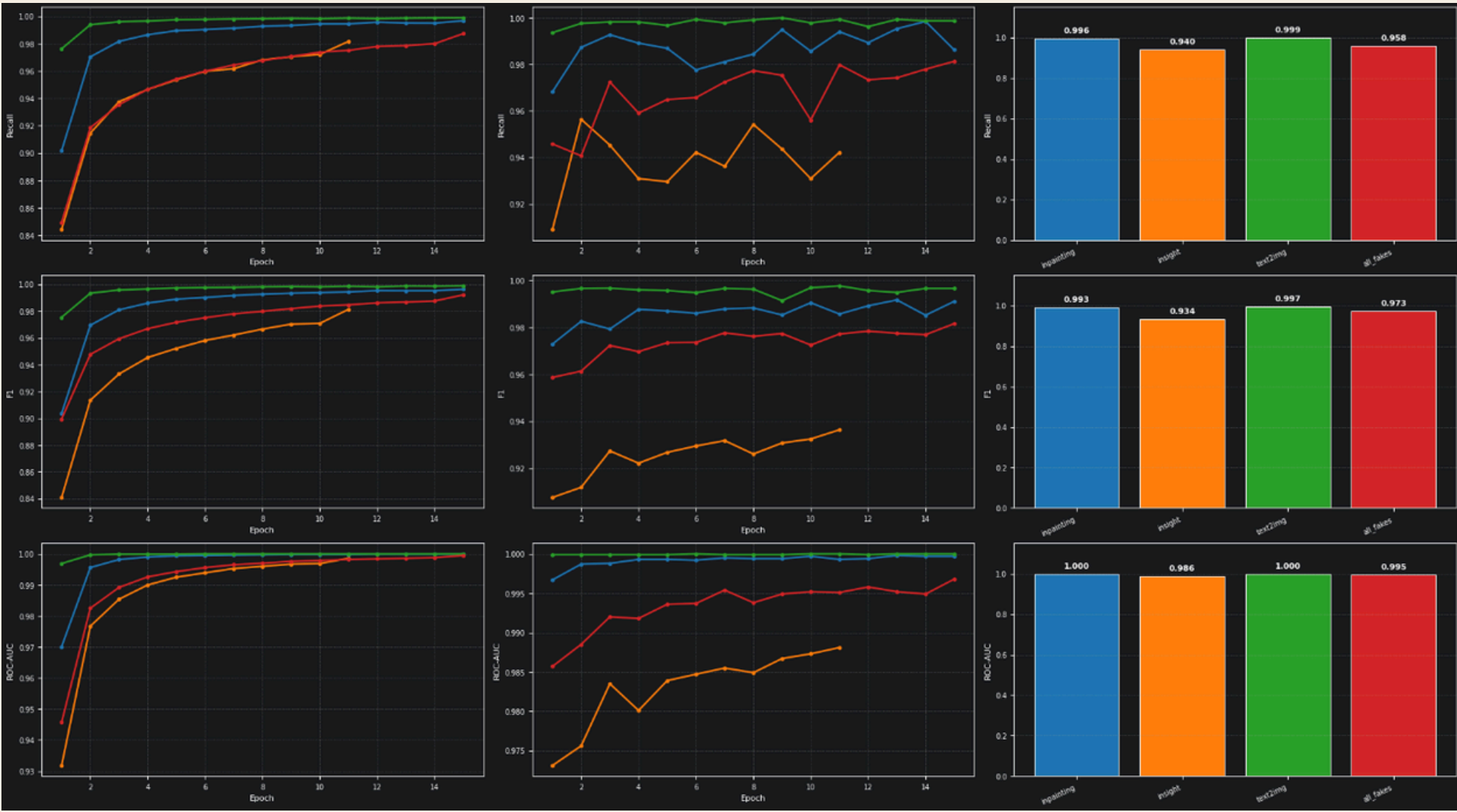


ViT_B_16 Full



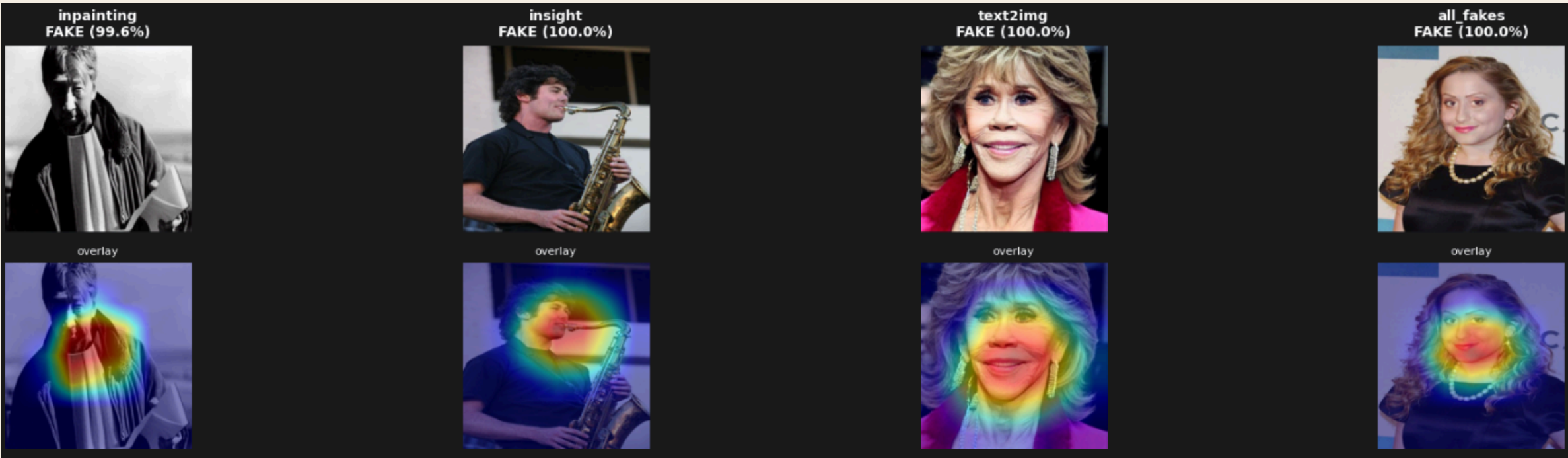
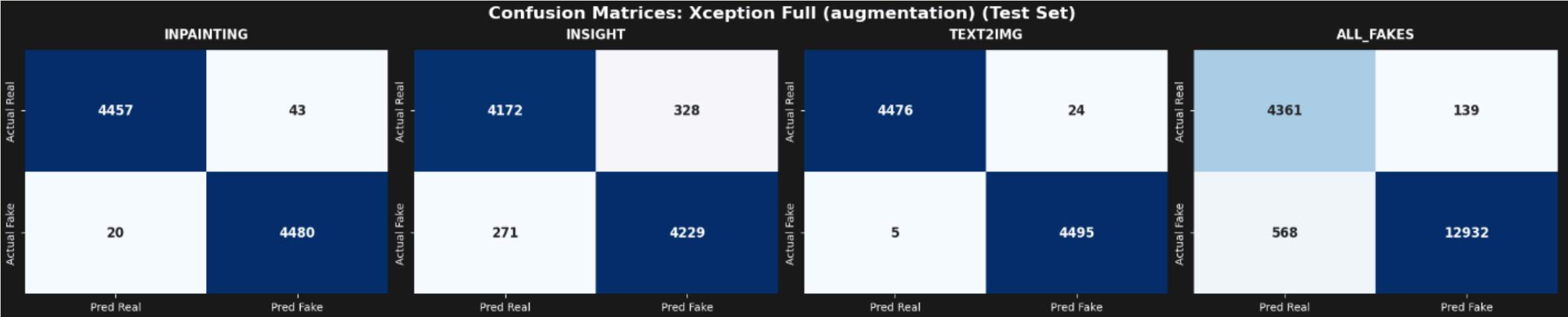
RESULTS BEST MODEL DEEP DIVE

XCEPTION FULL WITH AUGMENTATION



RESULTS BEST MODEL DEEP DIVE

XCEPTION FULL WITH AUGMENTATION



DISCUSSION & CONCLUSIONS

Full Fine-Tuning is Essential

Unfreezing all layers significantly outperformed frozen architectures. ImageNet pre-training alone was insufficient for this domain.

The Best Model

Selected for its superior interpretability. While quantitative metrics were similar across models, Xception showed the most precise, consistent focus on genuine facial features.

Augmentation Drives Interpretability

We accepted a slight drop in raw metrics to stop the models from "cheating" on background noise. Grad-CAM confirmed this forced a strict, trustworthy focus on actual faces.

The ViT Data Appetite

ViT underperformed top CNNs. Because it lacks built-in spatial inductive bias, it requires significantly larger datasets or longer training to learn spatial relationships.

DISCUSSION & CONCLUSIONS

Limitations

Generalization to unseen domains

The model classifies inpainting, insight, and text2img well, but how would it handle a completely new, unknown generation method?

Computational constraints

While Xception is highly accurate, its heavy computational cost may bottleneck real-time inference; production environments might require lighter models like EfficientNet, trading a fraction of accuracy for necessary speed.

DISCUSSION & CONCLUSIONS

Future Work

Ensemble Methods

Suggesting combining the spatial strengths of a CNN with the global context of a ViT.

Explainability

Going beyond Grad-CAM to more advanced interpretability methods to truly understand deepfake artifacts.

GENERATIVE

PROBLEM & APPROACH OVERVIEW

Problem: Generate realistic synthetic face images, trained on 30,000 real celebrity images from the wiki subset of DeepFakeFace.

The Strategy: Four models, each motivated by the failure of the previous one:

Order	Model	Key Idea
1	Simple DCGAN	Establish a baseline
2	Improved DCGAN	Fix discriminator dominance
3	WGAN-GP	Wasserstein loss + gradient penalty for stable training
4	DDPM	Abandon adversarial training entirely

Main evaluation metric: FID (Fréchet Inception Distance). Measures distributional similarity between real and generated images using Inception v3 features.

MODEL ARCHITECTURE & DESIGN CHOICES

Our first model - Baseline DCGAN

The generator takes a random noise vector (100 dimensions) and transforms it through a series of upsampling layers into a 128×128 image. The discriminator takes an image and outputs whether it thinks the image is real or fake.

Training alternates between the two: the discriminator learns to tell real from fake, and the generator learns to fool the discriminator. The hope is that this adversarial pressure pushes the generator to produce increasingly realistic images.

For this baseline we used a standard DCGAN setup:

- **Generator:** 5 ConvTranspose2d layers, BatchNorm + ReLU, Tanh output
- **Discriminator:** 5 Conv2d layers, BatchNorm + LeakyReLU(0.2), raw logit output
- **Optimizer:** Adam, $lr = 2 \times 10^{-4}$, $\beta_1 = 0.5$
- **Loss:** Binary Cross-Entropy (BCE)
- Epochs: 250 | Batch size: 64 | Latent dim: 100
- In Option 1 we deliberately **don't** use **Label smoothing**

MODEL ARCHITECTURE & DESIGN CHOICES

Improved DCGAN

Option 1 showed that the baseline DCGAN's main problem was discriminator dominance, it learned to separate real from fake too easily, leaving the generator with vanishing gradients. In this model we apply three standard techniques to address this:

- **Spectral Normalisation on the discriminator:** constrains the Lipschitz constant of each layer, preventing the discriminator from becoming arbitrarily confident. This replaces BatchNorm in the discriminator.
- **Label Smoothing (0.15):** instead of training the discriminator with hard labels (real=1, fake=0), we use real=0.85. This softens the discriminator's confidence and keeps the gradient signal alive.
- **Two Time-Scale Update Rule (TTUR):** the generator uses a higher learning rate ($2e-4$) than the discriminator ($1e-4$), giving the generator a head start in each training step.
- Epochs Runned: 100.

We also increase the generator capacity from $ngf=64$ to $ngf=128$, doubling the number of filters per layer so it can model finer facial details.

MODEL ARCHITECTURE & DESIGN CHOICES

WGAN-GP (Wasserstein GAN with Gradient Penalty)

Options 1 and 2 showed that BCE loss is fundamentally problematic for GAN training, it either saturates when the discriminator is too strong (Option 1) or produces a lazy equilibrium when the discriminator is constrained (Option 2).

This model replaces BCE with the Wasserstein distance. Provides smooth, non-saturating gradients regardless of discriminator strength.

The key changes from the Improved DCGAN (Option 2):

- **Wasserstein loss:** the critic maximises $E[C(\text{real})] - E[C(\text{fake})]$, the generator minimises $-E[C(\text{fake})]$. No log, no saturation.
- **Gradient Penalty (GP):** replaces weight clipping. Enforces a 1-Lipschitz constraint by penalising the critic when its gradient norm deviates from 1. This is computed on interpolations between real and fake images.
- **InstanceNorm instead of BatchNorm in the critic:** BatchNorm is incompatible with gradient penalty because GP requires per-sample gradients, while BatchNorm couples samples within a batch.
- **N_critic = 2:** the critic is trained 2 steps for every 1 generator step, giving it a more accurate distance estimate.

MODEL ARCHITECTURE & DESIGN CHOICES

DDPM - (Denoising Diffusion Probabilistic Model) with Face Detection

All three GAN variants shared the same fundamental challenge: training two networks adversarially requires careful balancing. The adversarial framework itself is the bottleneck.

We take a clean image (x_0) and directly calculate its noisy counterpart (x_t) at a randomly chosen timestep t in $[1, 1000]$ using the closed-form formula:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

- **The Noise Schedule:** A linear progression where variance scales from $\beta_1 = 10^{-4}$ up to $\beta_T = 0.02$ across the $T=1000$ steps.

The network (**U-Net**) is shown the noisy image and must predict the exact Gaussian noise that was injected.

- **The Loss Function:** Simple Mean Squared Error (MSE) between the predicted and actual noise.
- **The Advantage:** No adversary, no vanishing gradients, and no complex Nash equilibrium balancing act.

MODEL ARCHITECTURE & DESIGN CHOICES

DDPM - (Denoising Diffusion Probabilistic Model) with Face Detection

U-Net Architecture

The noise predictor is a U-Net with:

- Sinusoidal timestep embeddings injected into each residual block, telling the network how much noise to expect
- Channel progression: $64 \rightarrow 128 \rightarrow 192 \rightarrow 256$ through the encoder, mirrored in the decoder
- Self-attention at 16×16 and at the 8×8 bottleneck to capture long-range spatial relationships (e.g. symmetry between eyes).
- Skip connections between encoder and decoder at every resolution level
- GroupNorm throughout (more stable than BatchNorm for small batch sizes)
- 15M parameters

Face Detection Preprocessing

We use MTCNN (Multi-Task Cascaded Convolutional Networks) to detect and crop faces with a margin, then resize to 128×128 . This tightens the data distribution so the model can focus entirely on learning facial features. The face detection runs once as a preprocessing step before training begins.

RESULTS

Baseline DCGAN

FID: 241 $\rightarrow$ 133 (epoch 10 $\rightarrow$ 250) Still high

What went wrong - Discriminator Dominance:

- Discriminator loss $\rightarrow$ near 0; Generator loss climbs from ~ 3 to ~ 7
- $D(x) \approx 0.98$ (near-certain on reals), $D(G(z)) \approx 0.02$ (near-certain on fakes)
- In a healthy GAN both should hover around 0.5
- Generator receives vanishing gradients $\rightarrow$ improvement stalls after epoch ~ 50

Conclusion: Standard BCE saturates when the discriminator is too confident. We need to constrain it.

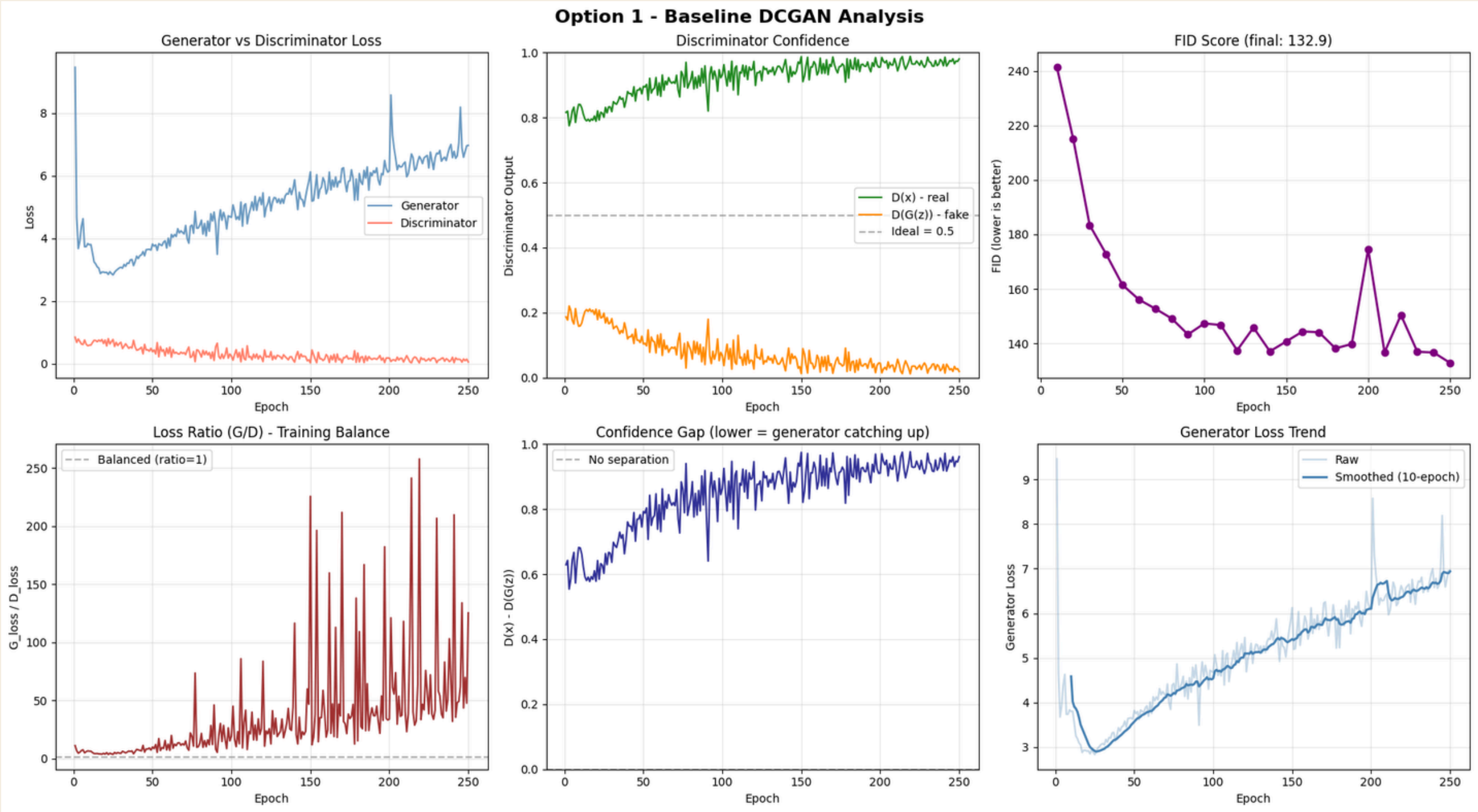


OPTION 1 - BASELINE DCGAN SUMMARY

```
=====
Epochs trained      : 250
Final G_loss         : 6.9713
Final D_loss         : 0.0556
Final D(x)           : 0.9804 (ideal: 0.5)
Final D(G(z))        : 0.0190 (ideal: 0.5)
Final loss ratio     : 125.4x
Best FID              : 132.86 (epoch 250)
Final FID             : 132.86
=====
```

RESULTS

Baseline DCGAN



RESULTS

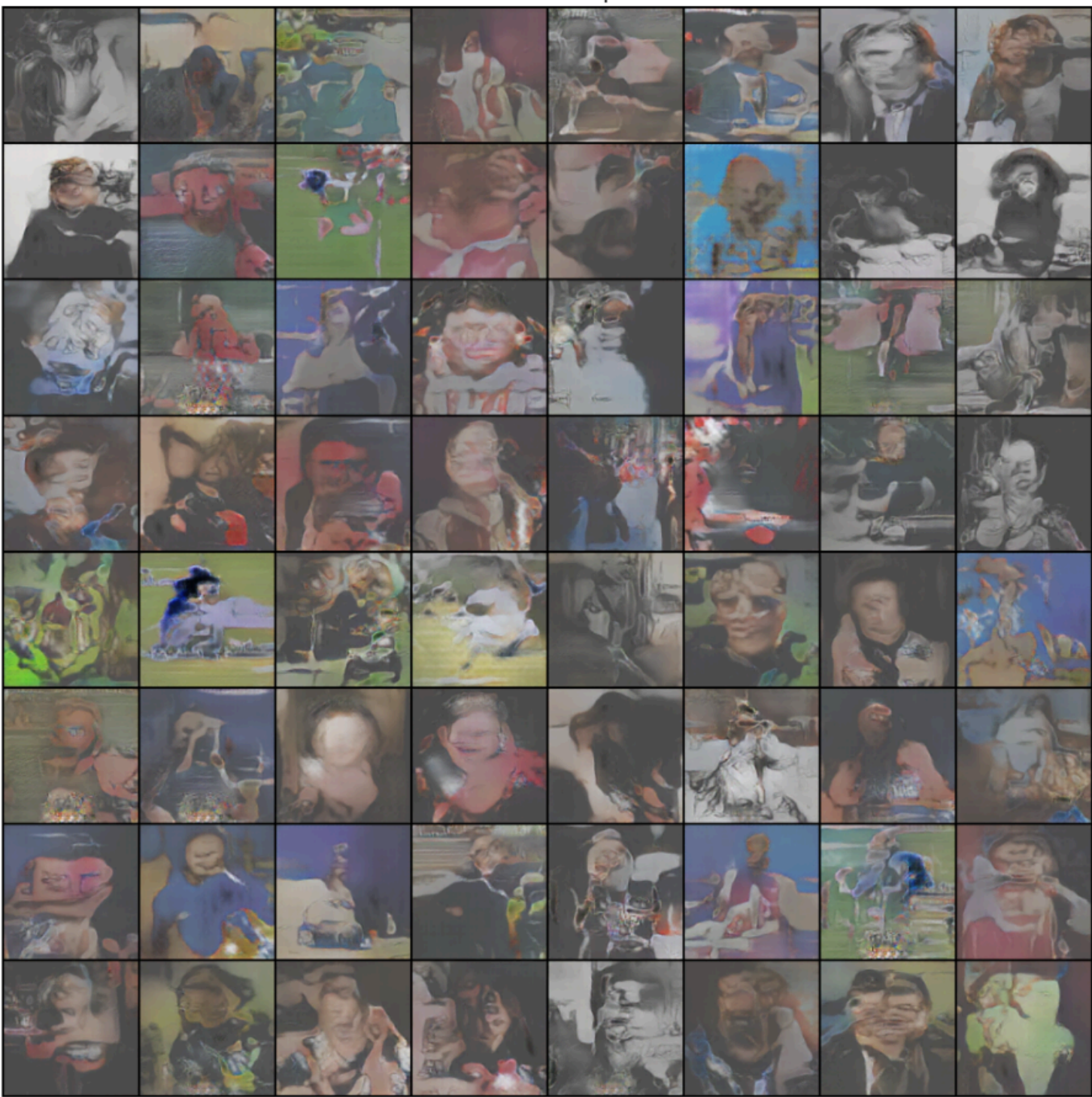
Improved DCGAN

FID: 204 Worse than Option 1 when both at epoch 100

The opposite problem - Discriminator Too Weak:

- Training is stable : no spikes, losses stay in a narrow band.
- But generator loss sits at exactly $0.70 \approx -\ln(0.5)$ throughout
- $D(G(z)) \approx 0.52$ from epoch 1 : discriminator is immediately fooled
- Generator learns to produce images that get 50/50 scores, not realistic ones
- Result: grey, desaturated, low-contrast images, lazy equilibrium

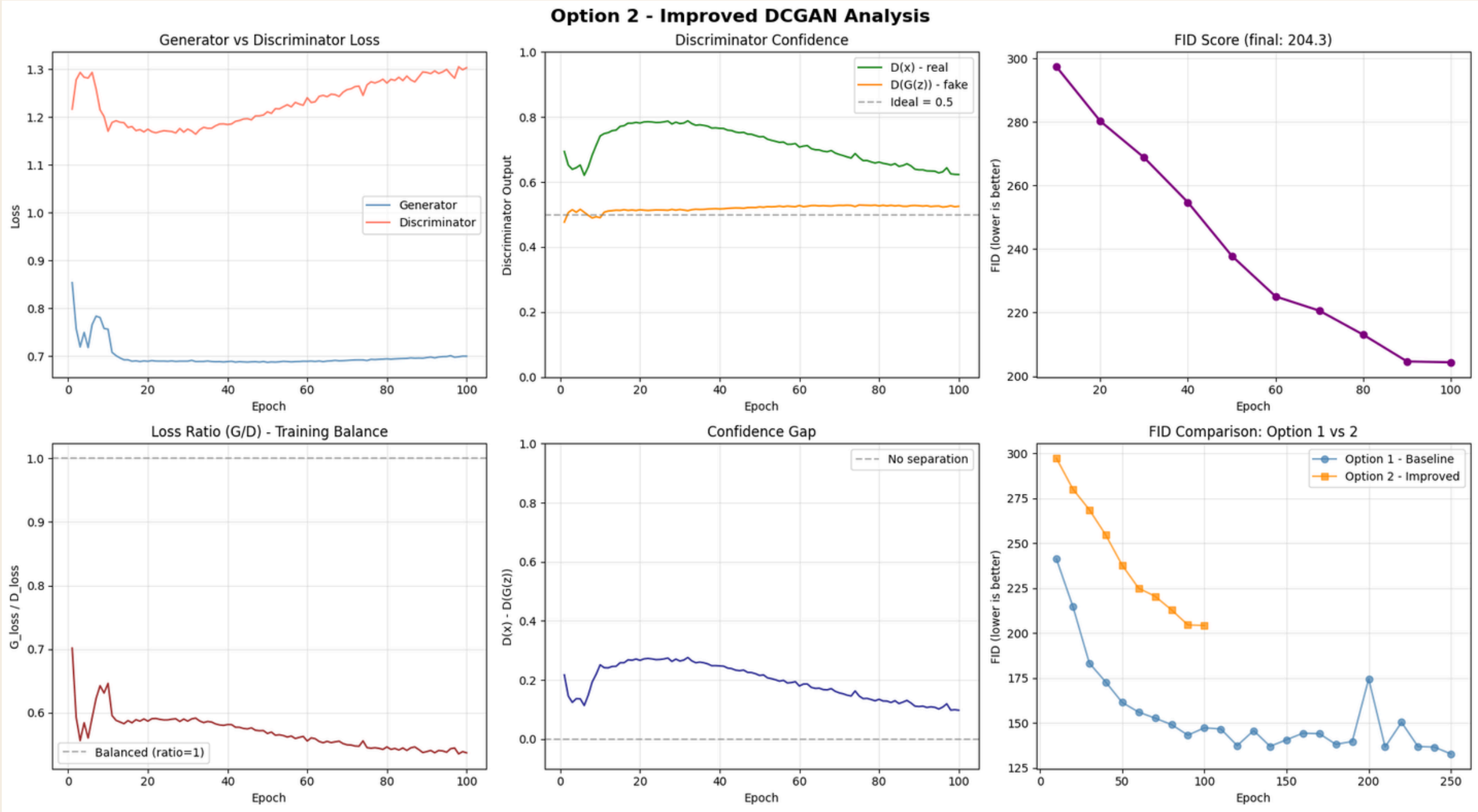
Conclusion: We constrained the discriminator too much. BCE loss has no middle ground, it either saturates or gives useless gradients. We need a fundamentally better loss function.



OPTION 2 - IMPROVED DCGAN SUMMARY			
Epochs trained	: 100		
Final G_loss	: 0.7002	(Option 1: 6.9713)	
Final D_loss	: 1.3035	(Option 1: 0.0556)	
Final D(x)	: 0.6232	(Option 1: 0.9804)	
Final D(G(z))	: 0.5254	(Option 1: 0.0190)	
Best FID	: 204.33	(epoch 100)	
Final FID	: 204.33	(Option 1: 132.86)	

RESULTS

Improved DCGAN



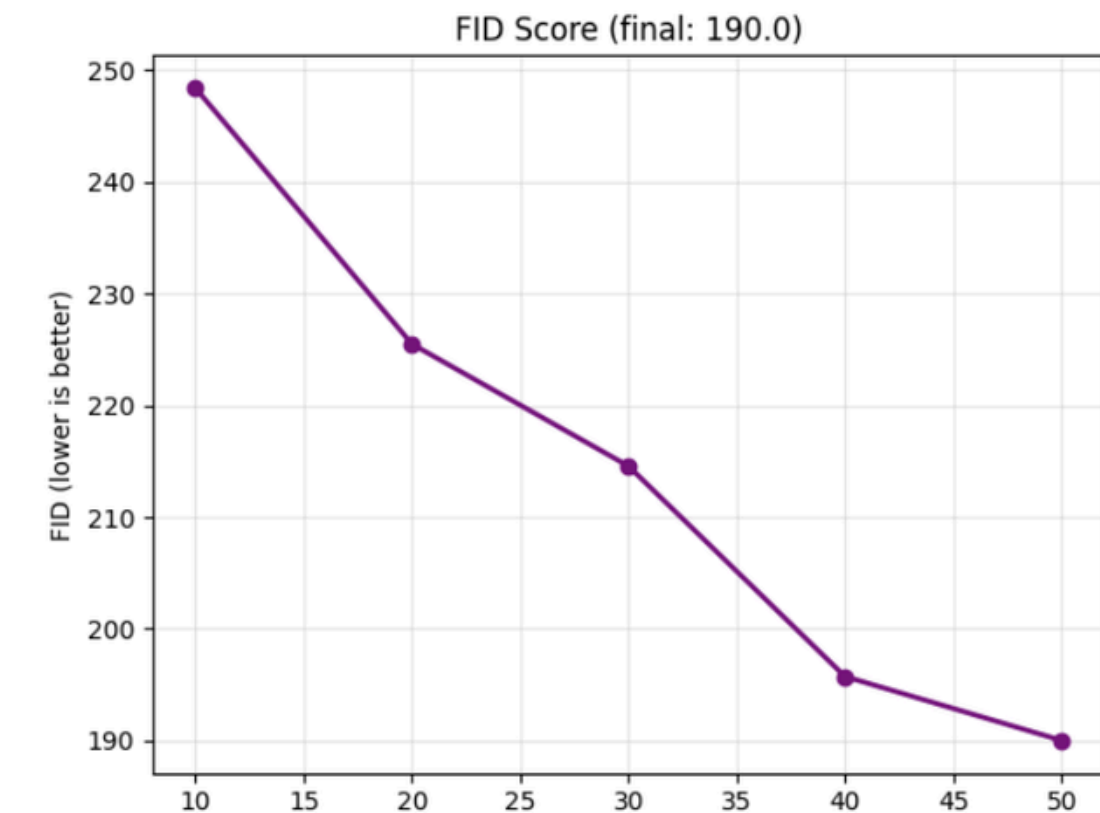
RESULTS

WGAN-GP

Despite the mathematical soundness of the Wasserstein loss, the final metric revealed a major limitation:

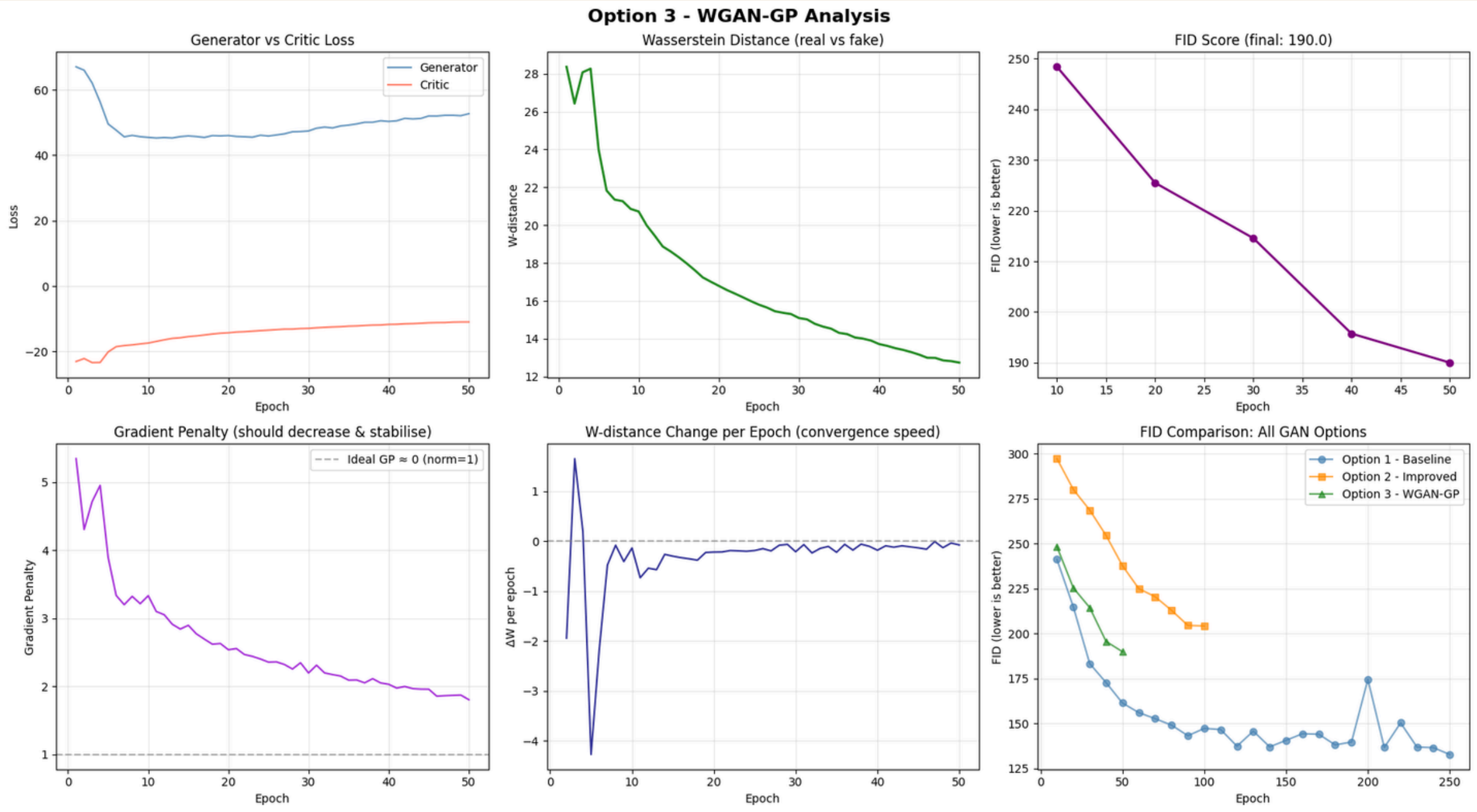
- **Underperforming the Baseline:** An FID of 190 means it failed to beat the baseline that had FID 160 at epoch 50.
- **The Penalty of Slow Convergence:** While WGAN-GP is exceptionally stable, it is notoriously slow to converge. At 50 epochs, the model was still in its early "learning phase." It could lay down the macro-geometry of a face but lacked the training time required to build proper facial features.

Since we had limited compute budget we tried other approaches instead of running more epochs since it took very long to run 50 epochs.



RESULTS

WGAN-GP



RESULTS

DDPM

FID: 122 → 64.4 (epoch 10 → 100) - Best by far

Why it works? No adversarial training:

- Single UNet trained with simple MSE loss (predict the noise added to an image).
- No discriminator to balance, no vanishing gradients, no mode collapse.
- Loss drops 74% (0.050 → 0.013) and converges smoothly in ~15 epochs.
- Face cropping with MTCNN (96% detection rate) tightened the data distribution, letting the model focus on faces instead of backgrounds.
- The result: **FID 64.4** roughly half of the best GAN (Option 1: 132.9).
- At epoch 50, DDPM already beats every GAN at any epoch count.

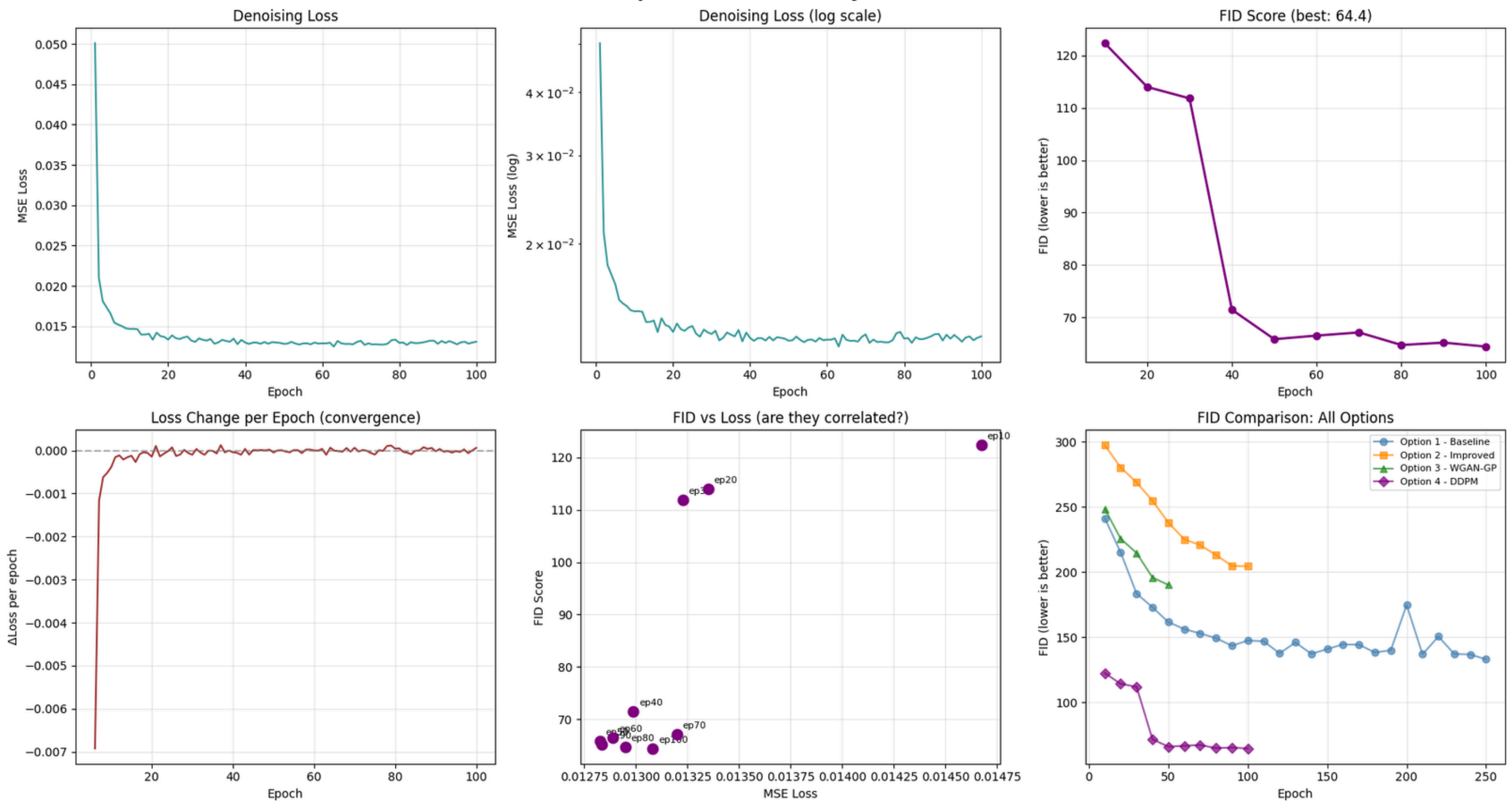
Conclusion: The adversarial framework was the bottleneck. Replacing it with a regression-based diffusion approach, combined with cleaner training data, solved the core instability problem and produced far better results in less time.



OPTION 4 - DDPM SUMMARY	
Epochs trained	: 100
Final loss	: 0.013081
Loss reduction	: 0.0501 → 0.0131 (74% decrease)
Best FID	: 64.41 (epoch 100)
Final FID	: 64.41
FID vs best GAN	: 64.4 vs 132.9 (Option 1)

RESULTS

Option 4 - DDPM Analysis



RESULTS

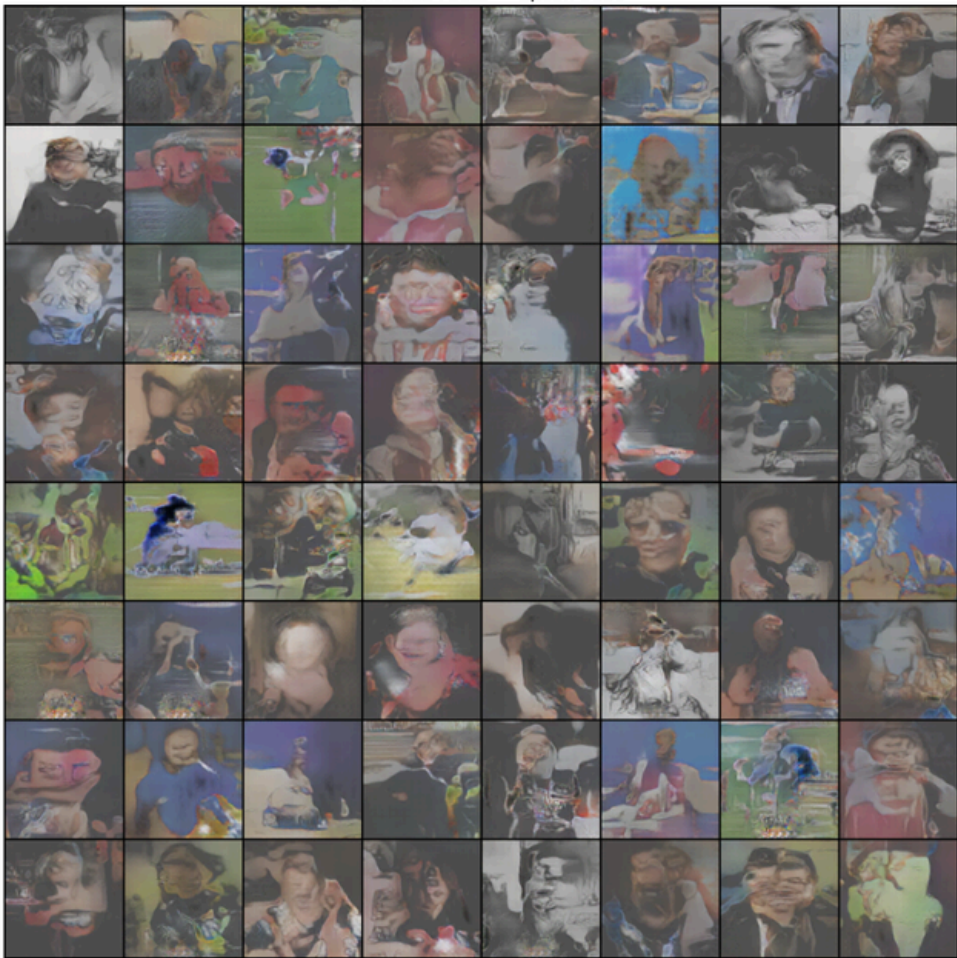
Option 1 - Final Epoch

Generated Faces — Epoch 9999



Option 2: Improved DCGAN - Final Epoch

Generated Faces — Epoch 9999



Option 3: Final Epoch

Generated Faces — Epoch 9999



DDPM - Final Generated Faces (Pre-rendered)

DDPM — Final Generated Faces (1000-step sampling)



DISCUSSION & CONCLUSIONS

Each model was motivated by the failure of the previous one:

- Option 1 → discriminator too strong → constrain it → Option 2
- Option 2 → discriminator too weak → replace the loss function → Option 3
- Option 3 → stable but too slow → abandon adversarial training → Option 4

The core problem was never the architecture, it was the adversarial training dynamics. Every fix to one side created a new imbalance on the other. DDPM eliminated this entirely by reformulating generation as a single-network denoising regression.

Two factors drove the DDPM's success: a fundamentally more stable training paradigm, and tighter data via MTCNN face cropping (28,786 / 30,000 images, removing noisy backgrounds).

Remaining gap to state-of-the-art (FID ~20–30): mainly due to our UNet being only 15M parameters (vs typical 60M+), limited by fitting all four experiments on a single GPU. A larger model, more epochs, and other techniques would likely close this gap.

15 May 2026

THANK YOU

Thanks For Watching

João Proença 202207835

Joana Pimenta 202206120